

Dynamic Programming — Complete Guide

1. What is DP?

Dynamic Programming is a technique for solving problems by breaking them into overlapping subproblems, solving each subproblem once, and storing (caching) the result so it's never recomputed.

Two properties must hold:

- **Optimal substructure** — the optimal solution to the problem can be built from optimal solutions to its subproblems.
- **Overlapping subproblems** — the same subproblem is solved again and again if you use plain recursion.

If a problem has optimal substructure but *no* overlapping subproblems (each subproblem is solved once), that's usually **Divide & Conquer**, not DP (e.g., merge sort).

2. Why is it used?

Plain recursion on problems with overlapping subproblems leads to exponential blow-up (e.g., naive Fibonacci is $O(2^n)$). DP converts this to polynomial time by trading space for time — you pay memory to store subproblem answers and reuse them.

3. Where is it used?

- Optimization problems: min/max cost, min/max profit, longest/shortest sequence
- Counting problems: number of ways to do X
- Decision problems: can you achieve X? (feasibility)
- Real-world: sequence alignment (bioinformatics), shortest paths (Bellman-Ford), text diff tools, resource allocation, scheduling, compilers (register allocation), speech recognition (Viterbi), finance (portfolio optimization)

4. How to identify a DP problem — the checklist

Ask these in order:

1. **Can I make a recursive brute-force solution?** If yes, does it explore a decision tree (choices at each step)?
2. **Do subproblems repeat?** Draw the recursion tree mentally for a small input — do you see the same (state) appearing more than once?
3. **Is there a "choose or not choose" / "take this option or that option" structure?**
Classic DP smell.
4. **Does the problem ask for optimal (min/max), count of ways, or yes/no feasibility** — over some sequence, grid, or set of choices?
5. **Can I define the answer to the whole problem in terms of the answer to a smaller version of itself?** (recurrence relation)

If you answer yes to 1, 2, and 4 — it's DP.

5. Can constraints hint at DP? — YES, this is the single fastest signal

This is the most reliable "2-minute" trick, especially in competitive programming / interviews:

Constraint (n)	Likely complexity expected	Hints
$n \leq 20$	$O(2^n)$	Bitmask DP
$n \leq 500$	$O(n^3)$	DP over pairs/intervals (e.g. matrix chain, interval DP)
$n \leq 5,000$	$O(n^2)$	DP over 2 indices
$n \leq 10^5 - 10^6$	$O(n \log n)$ or $O(n)$	1D DP, DP + binary search, DP + monotonic structure
Target sum / capacity given (e.g. "sum $\leq 10^4$ ", "weight ≤ 1000 ")	$O(n \times \text{target})$	Classic knapsack-style DP — the target size itself becomes a dimension of your DP table

Rule of thumb: If the problem gives you two numeric bounds (like n and $\text{target} / \text{capacity} / w$), and their product is reasonable ($\leq \sim 10^7 - 10^8$), that's almost a signature of DP — the second bound is telling you what your second dimension should be.

Also: if brute force is clearly exponential (recursion / all subsets / all partitions) AND n is small-ish ($\leq \sim 40$), that's DP-or-backtracking-with-memo territory.

6. The "2-minute" pattern triggers (keywords → DP)

Train yourself to pattern-match on phrasing:

- "minimum/maximum number of ways", "count the number of ways"
- "minimum cost / maximum profit to reach..."
- "longest / shortest subsequence / substring / subarray"
- "can you partition / can you make exactly..." (subset sum flavor)
- "at each step you can choose..." (decision at every index)
- "find if it's possible to reach the end" with jumps/steps
- Grid problems: "paths from top-left to bottom-right"
- "edit distance", "longest common subsequence"
- Anything phrased recursively: " $f(n)$ depends on $f(n-1)$ and $f(n-2)$ "
- Interval-based: "minimum cost to merge/burst/cut..."

If the problem *sounds like* "brute force = try every combination/every split point," and asks for optimal/count — DP.

7. Intuition / rationale

Think of DP as "**smart brute force**." You still consider all possibilities conceptually (recursion covers the full search space), but:

- You **cache** answers to subproblems (state) so identical work isn't redone → memoization (top-down).
- Or you **build up** answers from the smallest subproblem to the largest, in an order that guarantees dependencies are ready → tabulation (bottom-up).

The core mental model: **define a "state"** (a small set of parameters that fully describes a subproblem) and a **recurrence** (how the answer for a state is built from answers to smaller states). Everything else is implementation.

8. Types of DP (with recognition patterns)

Type	Recognize by	Example problems
1D DP	Answer depends on previous 1–2 states in a linear sequence	Fibonacci, climbing stairs, house robber
2D DP (grid)	Movement/paths on a grid	Unique paths, min path sum
2D DP (two sequences)	Comparing two strings/arrays	LCS, edit distance
Knapsack (0/1)	Item + capacity, each item used once	Subset sum, partition equal subset
Unbounded Knapsack	Items reusable infinitely	Coin change, rod cutting
Interval DP	Choose a split point in a range $[i, j]$	Matrix chain multiplication, burst balloons, palindrome partitioning
Bitmask DP	Small n ($\leq \sim 20$), track subset of used elements	TSP, assignment problems
Digit DP	Count numbers with property, bounded by a number's digits	Count numbers $\leq N$ with digit sum X
Tree DP	Recurrence over a tree's children	House robber III, max path sum in tree
DP on states/graphs	Explicit "state machine" (e.g. holding stock or not)	Best time to buy/sell stock with cooldown
DP + Binary Search	$O(n \log n)$ LIS-style optimization	Longest increasing subsequence
Probability/Expectation DP	"expected number of..."	Dice/board game problems

9. Reusable templates

Top-down (memoization) — usually the easiest to *derive* from brute force

```
from functools import lru_cache
```

```

def solve(input_data):
    @lru_cache(maxsize=None)
    def dp(state):
        # base case(s)
        if base_condition:
            return base_value

        # try all choices at this state
        best = default_value # 0, inf, -inf, etc.
        for choice in choices_available(state):
            result = compute(choice) + dp(next_state(state, choice))
            best = combine(best, result) # min/max/sum
        return best

    return dp(initial_state)

```

Bottom-up (tabulation) — generic 1D

```

def solve(n):
    dp = [0] * (n + 1)
    dp[0] = base_case_0
    dp[1] = base_case_1 # if needed

    for i in range(2, n + 1):
        dp[i] = combine(dp[i - 1], dp[i - 2]) # recurrence

    return dp[n]

```

Bottom-up — generic 2D (grid / two sequences)

```

def solve(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[n][m]

```

0/1 Knapsack template

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i-1][w] # don't take item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1])

    return dp[n][capacity]
```

General derivation workflow (use this every time)

1. Write brute-force recursion first (no memo). Get it correct.
2. Identify the **state**: what varies between recursive calls? (usually index, remaining capacity/sum, last-taken element, etc.)
3. Add memoization on that state (dict or array cache).
4. If needed, convert to bottom-up for better constants / to avoid recursion depth limits.
5. Optimize space: if `dp[i]` only depends on `dp[i-1]` (or last k rows), collapse to rolling variables.

10. Time & space complexity (generic)

- **Time** $\approx$ (number of distinct states) $\times$ (work done per state)
 - 1D DP: $O(n)$ states $\times O(1)$ transition $\rightarrow O(n)$
 - 2D DP: $O(n \cdot m)$ states $\times O(1)$ transition $\rightarrow O(n \cdot m)$
 - Interval DP: $O(n^2)$ states $\times O(n)$ transition (trying split points) $\rightarrow O(n^3)$
 - Bitmask DP: $O(n \cdot 2^n)$ states $\times O(n)$ transition $\rightarrow O(n^2 \cdot 2^n)$
- **Space** $\approx$ number of states stored, i.e., size of the dp table (can often be reduced by one dimension via rolling arrays, e.g. $O(n \cdot m) \rightarrow O(m)$).

11. Quick self-test flow (do this in your head in ~2 minutes)

1. Read constraints first $\rightarrow$ does n suggest $O(n^2)$, $O(n \cdot \text{target})$, $O(2^n)$? $\rightarrow$ strong DP hint.

2. Does the problem ask min/max/count/possible? → DP smell.
3. Can I describe a state with 1–3 variables (index, remaining capacity, last choice) such that the answer at a bigger state is built from smaller states? → if yes, it's DP, and you've basically already found your recurrence.
4. If states don't compress into a few variables (e.g., truly need to remember an entire arbitrary subset with no small n), it might not be DP — could be greedy, graph, or NP-hard.